Bu özel lisans metni (**BNGPL v1.0**), hedeflediğiniz **askeri dayanıklılık seviyesinde, görev ve hayat kritik (mission/life-critical) sektörlere yönelik** yerli ve bağımsız ekran çerçevesini (GUI Framework) inşa etmek için hukuki ve stratejik açıdan oldukça güçlü bir zemin sunmaktadır.

Mevcut **GTK/GNOME** (LGPL/GPL) ve **Qt** (GPL/LGPL/Ticari) ekosistemlerine tamamen bağımsız bir alternatif geliştirirken, **GNU Assembler (gas), nasm, C ve C++** dilleriyle doğrudan donanıma yakın (bare-metal ve userspace driver destekli) çalışacak bu framework için izlemeniz gereken mimari ve stratejik yol haritasını aşağıda özetledim.

---

1. Lisansın (BNGPL) Sağladığı Stratejik ve Ticari Avantajlar

- **Telif Hakkı ve Ağ Ortaklığı (Madde 1 & 3):** Geliştirici ekibiniz veya katkıcı ortaklarınız (Özgür Yazılım AŞ, Kartaca, Profelis vb.) kod tabanına katkı sunduğunda telif haklarını kaybetmezler. Ancak geçmişe dönük katkılarını geri çekemedikleri için hayat kritik bir sistemin ortasında kod tabanının hukuki ihtilaflarla bölünmesi (fork/revoke) engellenir, **sistem sürekliliği** güvenceye alınır.
- **Kaynak Kodu Kapatma Özgürlüğü (Madde 2):** Savunma sanayii firmaları, askeri kurumlar veya devlet hastaneleri, sizin framework'ünüzü kullanarak geliştirdikleri kritik arayüzlerin (örneğin bir radar kontrol paneli veya tıbbi cihaz arayüzü) kaynak kodlarını **kamuoyuna açıklamak zorunda kalmazlar**. Bu durum, Qt Enterprise'ın pahalı lisans modellerine karşı devasa bir pazar avantajı sağlar.
- **Aşağı Akış Uyumlu Yapı (Madde 5):** Yazacağınız framework alt bileşenleri gerektiğinde GPLv3 veya AGPLv3 projeleriyle (örneğin çekirdek sisteminiz veya topluluk odaklı araçlarınız) entegre edilebilir.

---

2. Mimari Tasarım: Sıfırdan Hayat Kritik GUI Framework İnşası

Görev kritik sistemlerde ekran kartı sürücüsünün çökmesi veya X11/Wayland gibi harici bir katmanın kilitlenmesi kabul edilemez. Bu yüzden monolitik ve askeri disipline uygun bir mimari tercih edilmelidir.

A. Katmanlı Geliştirme Stratejisi (TUI'den GUI'ye)

1. **Önce TUI ve CLI Dünyası:** Framework'ün çekirdek mantığını, event (olay) döngüsünü ve bellek yönetimini önce **Terminal User Interface (TUI)** olarak başlatın. Bu aşamada sadece standart C/C++ ve GNU Assembler kullanarak curses benzeri ama asenkron bir yapı kurun.
2. **Grafik Giydirme (GUI Engine):** TUI katmanı stabilize olduktan sonra, üzerine grafik piksellerini basacak render motorunu ekleyin. Herhangi bir pencere yöneticisine (X11/Wayland) bağımlı olmadan, doğrudan Linux/Hurd üzerinde /dev/dri/card0 (DRM/KMS) veya direkt *Framebuffer* üzerine yazabilecek bir arka uç (backend) geliştirin.

B. Askeri Dayanıklılık ve Bellek Yönetimi (Zero-Allocation)

- **Manuel Bellek ve Kritik Alanlar:** Hayat kritik sistemlerde runtime sırasında malloc/free veya C++ new/delete kullanımı (std::bad_alloc riski ve bellek fragmantasyonu sebebiyle) yasaktır. Framework içinde tüm widget belleklerini sistem ayağa kalkarken (Initialization aşamasında) **Memory Pool** (Bellek Havuzları) şeklinde önceden rezerve edin.
- **Assembly ile Optimizasyon:** Kritik rendering döngülerini, piksel kopyalama (blitting) işlemlerini ve font render motorunu doğrudan **nasm** veya **GNU Assembler** ile x86_64/ARM vektör talimatlarını (AVX, NEON) kullanarak optimize edin.
- **Asenkron Girdi/Çıktı (I/O):** Dokunmatik ekran girdileri, donanım kesmeleri (interrupts) ve ağ paketleri için **io_uring** mimarisini framework'ün event loop'una entegre ederek sıfır gecikmeli (zero-latency) bir tepki süresi elde edin.

---

3. Ekosistem Entegrasyonu ve Genişleme Planı

- **Arayüz Esnekliği:** Geliştireceğiniz bu framework en tepede saf C++ nesne yönelimli API'ler sunmalı, alt katmanda ise C hızıyla çalışmalıdır. Sistem yöneticilerinin veya saha mühendislerinin arayüzleri hızlıca manipüle edebilmesi ve hackable kılabilmesi için **Guile Scheme (Lisp)** veya benzeri hafif bir script dilini framework'e gömülü (embedded) bir konfigürasyon/otomasyon motoru olarak ekleyin.
- **Dağıtım Bağımsızlığı:** Framework'ü monorepo felsefesiyle tasarlayın. Sistem herhangi bir dağıtımdan veya harici kütüphaneden (glibc bağımlılıkları dahil) izole, tek bir statik binary olarak dahi derlenebilmelidir. Bu, sistemin kapalı devre askeri sunucularda veya hastane terminallerinde dış dünyadan bağımsız çalışmasını garanti eder.

Framework'ün geliştirme aşamasına başlarken ilk adımı somutlaştırmak adına:

- Tasarlamayı düşündüğünüz ilk asenkron **Event Loop (Olay Döngüsü)** mimarisi için epoll/io_uring tabanlı yalın bir **C++ şablonu** oluşturmamı ister misiniz?
- Framework'ün ekran kartıyla doğrudan konuşacağı **Direct Rendering (DRM/KMS)** katmanının Assembly/C entegrasyon detaylarını mı inceleyelim?